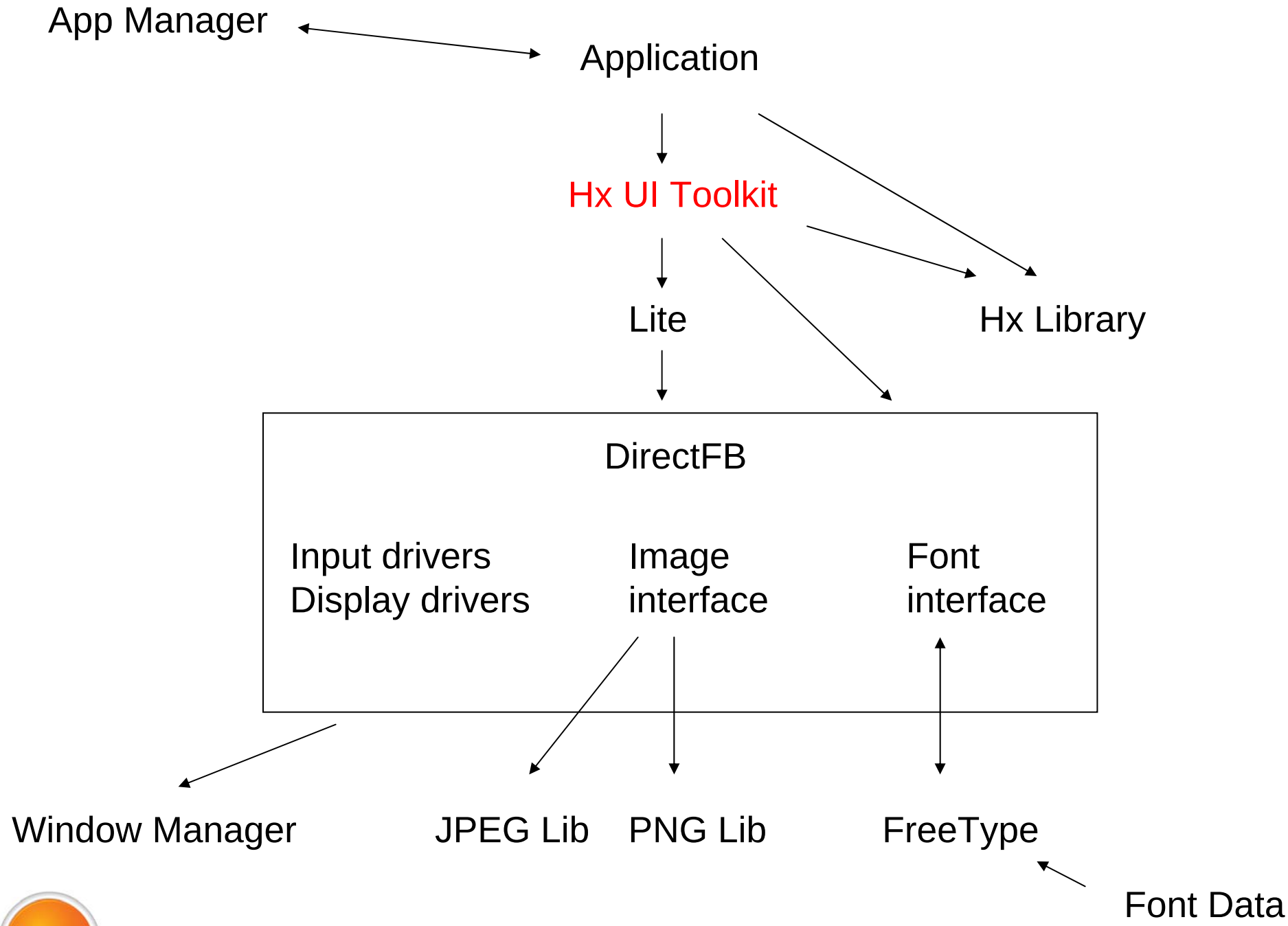
UI Design Principles & Concepts

Andy Stewart
Sr. Software Engineer
Phone Companion Team

UI Toolkit Design

- System Overview
- Toolkit Goals
- Key Concepts
- Application Guidelines


App Manager
Application
Hx UI Toolkit
Lite
Hx Library
DirectFB
Input drivers
Display drivers
Image
interface
Font
interface
Window Manager
JPEG Lib
PNG Lib
FreeType
Font Data

Toolkit Goals

- Fast implementation suitable for embedded system
  - Enable PDA behavior -- applications perceived as always running
  - Quick application launch (< 1 sec) and switch (< 400 ms)
- Consistent and simple API
  - Zen of Palm, and make it easy for application to customize
- Full screen environment
  - Single application window
  - Normally, application bar and system bar windows at top
- Emphasize scroll wheel and keyboard navigation
  - Wheel, Enter, Space, Arrow keys always interact with widgets

Windows and Widgets

- Surfaces
  - Heavyweight objects that includes the pixel buffer
  - `HxWindow_t` (onscreen buffer)
  - `HxImage_t` (offscreen buffer)
- Subsurfaces
  - Lightweight objects that define graphic context for an area
    - Color, font, clipping rectangle
  - `HxWidget_t`
  - `HxContainer_t` (a widget for grouping widgets)
    - Create views by hiding and showing containers

Update Model

Not necessary to update when dialogs or menus close

Dirty regions accumulated by user actions

- Regions are opaque, initialized with rectangle
- `HxWidgetUpdate` allows applications to explicitly define dirty region

On Update, window and widgets' Draw callbacks executed

- Draw begins at window, traversing widget hierarchy
- `HxWidgetDraw` allows applications to explicitly execute callback
- Drawing is done on Window Manager's back buffer, with Window Manager controlling window compositing and transfer of data from back buffer to display

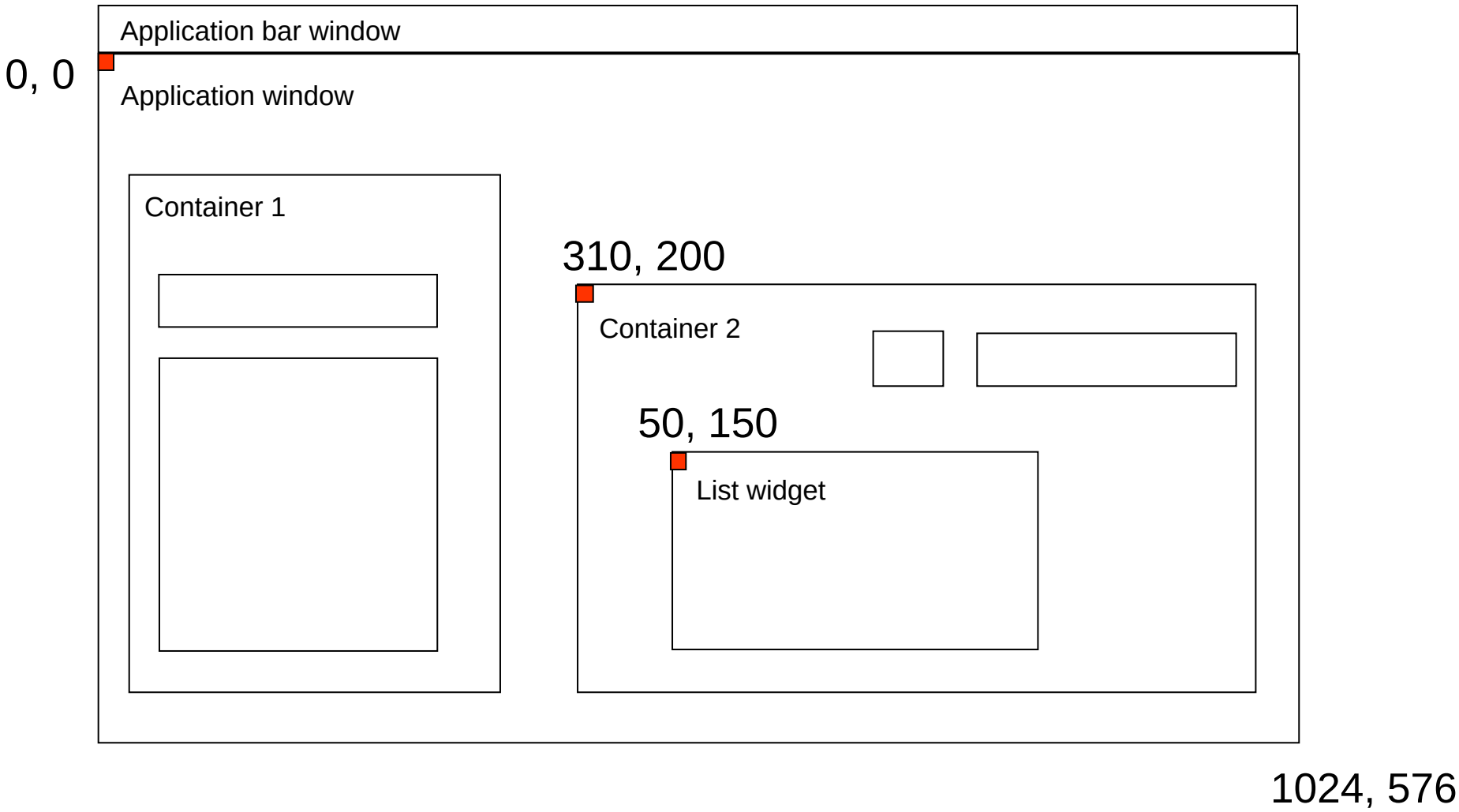
Event Loop

```
while (window && event loop active)
 if (Has event)
       Get event
       Handle event
 Update window
 Call timer callback
 Call idle callback
 Wait for event or timeout
```

Event Callbacks

- Widget event handlers called, followed by Window event handler
- Input events handlers are normally implemented by toolkit widgets with widget sending the appropriate message to applications
- Applications can override event handler using `HxWidgetSetCallback` and `HxWindowSetHandler`
  - If widget or window event callback is overridden, normally call system handler before exiting
- Widget state change callbacks (enable, visible, focus, resize)

Widget Hierarchy



Widget bounds

- Widget Bounds is the widget's footprint on display
  - Includes the area used by system for frame and focus ring
  - Hit testing by event loop based on this area
  - `HxWidgetGetBounds`
- Client Area does not include the system framing
  - `HxWidgetGetClientBounds` typically returns origin of (2, 2)
- Take footprint into consideration when laying out widgets

Application Guidelines
Techniques for fast application launch and switch

- Show and Hide (don't create and destroy)
  - `HxWidgetSetVisible`
  - `HxWindowSetOpacity`
- One technique t provide visual feedback to user ASAP
  - Create initial view
  - Enter event loop briefly to allow update
  - On exit, create additional views
  - Re-enter event loop forever

- Avoid large hxui files
- Build icons into applications
  - use `directfb-csource` and `mkdfiff`
- Keep mouse motion handlers quick

Application Guidelines
Analyzing performance

- Use prelinking before measuring launch time
- Test responsiveness with release build
- Use oprofile

Application Guidelines

- No concept of “Save” or “Quit”
  - PDA behavior
  - Applications need to save state when
    - Moved to the background
      - Close all dialogs on app switch (`kHxApplicationIsDeactivated` message)
      - Save dialog state if appropriate with `HxDialogCloseAllDialogs`

- Minimize the need to use trackpoint
  - Always provide menu shortcuts for gadgets in app bar or toolbar